

Regression Analysis and Optimization for the Runge Function

Zejing Wang

Zhenwei Liu

FYS-STK4155 – Project 1, Autumn 2026

Contents

1	Abstract	3
2	Introduction	3
3	Theory	3
3.1	a) OLS	3
3.2	b) Ridge	4
3.3	e) Gradient Descent	4
3.3.1	Plain Gradient Descent	4
3.4	f) Advanced gradient methods	4
3.4.1	Momentum	4
3.4.2	RMSprop	5
3.5	h) Stochastic Gradient Descent	5
3.6	g) Lasso	6
3.7	d) Cross-validation	6
3.8	c) Bias-Variance Trade-off	6
4	Methodology	7
4.1	Dataset	7
4.2	Polynomial Design Matrix	8
4.3	Ordinary Least Squares	8
4.4	Ridge Regression	8
4.5	Gradient Descent Optimizers	9
4.5.1	Plain Gradient Descent	9
4.5.2	Momentum	9
4.5.3	AdaGrad	9
4.5.4	RMSprop	10
4.5.5	Adam	10
5	Results	11
5.1	OLS and Ridge Regression	11
5.2	Bias-Variance Analysis	12
5.3	Cross-validation	13
5.4	Gradient Descent	13
5.5	Optimization Methods	14
5.6	Lasso Regression	15
5.7	Final model comparisons	16
6	Discussion	17
7	Conclusion	18

A	Use of AI/LLM Tools and references	19
A.1	Tools Used	19
A.2	Text Writing and Editing	19
A.3	Code Generation and Assistance	19
A.4	Code availability	19
B	References	19

1 Abstract

In this project, we study polynomial regression using the Runge function as a test problem. We apply Ordinary Least Squares (OLS), Ridge regression, and Lasso regression and investigate how the polynomial degree and regularization affect the model performance. The models are evaluated using the mean squared error (MSE) and the R^2 score.

Furthermore, we also implement various gradient descent techniques, including Plain Gradient Descent, Momentum, AdaGrad, RMSprop, and Adam to compare their advantages, disadvantages, and convergence behavior. Additionally, we implement Stochastic Gradient Descent (SGD) and investigate the effects of the number of epochs, batch size, and learning-rate schedule. We discuss why SGD can be useful compared with full-batch gradient descent and compare their computational costs.

Bootstrap and cross-validation (CV) are used to study the bias-variance trade-off and evaluate the generalization performance of the regression models. OLS, Ridge, and Lasso regression are compared using cross-validation. OLS and Ridge give almost identical minimum cross-validation errors, with a 10-fold CV MSE of approximately 0.0111, while Lasso gives a higher minimum CV MSE of approximately 0.0196. Ridge provides better stability for high-degree polynomial models, while the bias-variance analysis shows that increasing the amount of data reduces variance and improves model stability.

2 Introduction

Regression is a commonly used method in machine learning for modeling the relationship between input data and continuous target values. In this project, we use polynomial regression to approximate the Runge function

$$f(x) = \frac{1}{1 + 25x^2} \quad x \in [-1, 1] \quad (1)$$

We first generate noisy data using normal distribution, which we add to the Runge function. Then we divide the data into training and test sets. Ordinary Least Squares (OLS) and Ridge regression are then used to fit polynomial models of different degrees. Their performance is evaluated using MSE and the R^2 score. We then implement gradient descent for OLS and Ridge regression. The gradients are computed analytically and using automatic differentiation in PyTorch, and the obtained solutions are compared with the corresponding closed-form solutions. We further compare Plain Gradient Descent, Momentum, AdaGrad, RMSprop, and Adam in order to study how different optimization methods affect the convergence behavior. Bootstrap resampling is used to investigate the bias-variance trade-off, while k -fold cross-validation is used to estimate generalization error and select polynomial degree and regularization parameters. Lasso regression is also studied to investigate the effect of L_1 regularization and coefficient sparsity. Finally, Stochastic Gradient Descent is implemented using mini-batches, and batch size, number of epochs, and the learning-rate schedule are used to measure the effects. The final comparison uses cross-validation to compare OLS, Ridge, and Lasso regression on the same Runge-function problem.

3 Theory

3.1 a) OLS

Ordinary Least Squares (OLS) is used to find the parameters that minimize the difference between the predicted values and the true values. We first define the cost function as

$$C(\theta) = \frac{1}{n} \|X\theta - y\|^2 \quad (2)$$

Here, X is the design matrix, θ contains the model parameters, and y contains the true values.

The gradient of the OLS cost function is

$$\nabla C(\theta) = \frac{2}{n} X^T (X\theta - y) \quad (3)$$

The closed-form solution for OLS is

$$\theta_{OLS} = (X^T X)^{-1} X^T y \quad (4)$$

In practice, the pseudo-inverse or singular value decomposition (SVD) can also be used to calculate the solution.

3.2 b) Ridge

Ridge regression is similar to OLS, but it adds a regularization term to the cost function. The purpose of the regularization is to reduce large parameter values and reduce overfitting.

The Ridge cost function used in this project is

$$C(\theta) = \frac{1}{n} \|X\theta - y\|^2 + \frac{\lambda}{n} \|\theta\|^2 \quad (5)$$

where λ is the regularization parameter.

The gradient is

$$\nabla C(\theta) = \frac{2}{n} X^T (X\theta - y) + \frac{2\lambda}{n} \theta \quad (6)$$

The closed-form solution for Ridge is

$$\theta_{Ridge} = (X^T X + \lambda I)^{-1} X^T y \quad (7)$$

A larger value of λ gives stronger regularization and makes the parameters smaller.

3.3 e) Gradient Descent

Gradient descent is an iterative method for finding the parameters that minimize the cost function, i.e. optimizing the parameters. Instead of calculating the solution directly, the parameters are updated step by step.

The general update rule is

$$\theta_{k+1} = \theta_k - \eta \nabla C(\theta_k) \quad (8)$$

where η is the learning rate. The learning rate controls the size of each update. If its too small, the algorithm can converge very slowly, and if its too large, the algorithm may not converge (and may return NaN values numerically).

In this project, we calculate the gradients both analytically and with automatic differentiation and compare the results.

3.3.1 Plain Gradient Descent

Plain gradient descent directly uses the gradient (as in **3.3**) to update the parameters

$$\theta_{k+1} = \theta_k - \eta g_k \quad (9)$$

where

$$g_k = \nabla C(\theta_k)$$

The same learning rate is used during optimization.

3.4 f) Advanced gradient methods

Although plain gradient descent is a simple way to update the parameters, other gradient descent methods include momentum, AdaGrad, RMSprop and Adam.

3.4.1 Momentum

Momentum uses information from previous updates in addition to the current gradient. This can make gradient descent converge faster and reduce oscillations.

The update uses a velocity component, which is given as

$$v_{k+1} = \gamma v_k + \eta g_k$$

then we get parameter update

$$\theta_{k+1} = \theta_k - v_{k+1} \quad (10)$$

Here, γ controls how much of the previous update is kept.

3.4.2 RMSprop

RMSprop uses an adaptive learning rate. It adjusts the learning rate in each parameter. And it keeps a moving average of the squared gradients

$$s_k = \rho s_{k-1} + (1 - \rho) g_k^2$$

The parameters are updated using

$$\theta_{k+1} = \theta_k - \frac{\eta}{\sqrt{s_k} + \epsilon} g_k \quad (11)$$

The value ϵ is a small number used to avoid division by zero.

Adam

Adam combines ideas from Momentum and RMSprop. It keeps moving averages of both the gradient and the squared gradient.

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k$$

and

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2$$

we can then find their biases

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^t} \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^t}$$

and we can update the parameters as followed

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon} \quad (12)$$

Adam can adapt the step size for each parameter and often converges faster than plain gradient descent.

AdaGrad

AdaGrad is an adaptive optimization method that changes the learning rate for each parameter based on the previous gradients. It accumulates the squared gradients,

$$G_k = G_{k-1} + g_k^2$$

The parameters are then updated using

$$\theta_{k+1} = \theta_k - \frac{\eta}{\sqrt{G_k} + \epsilon} g_k \quad (13)$$

This reduces the effective learning rate for parameters that have received large gradients.

3.5 h) Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is different from normal gradient descent because it does not use the complete training data for every update. Instead, the training data is divided into smaller mini-batches.

For each mini-batch, the parameters are updated using

$$\theta_{k+1} = \theta_k - \eta \nabla C_B(\theta_k) \quad (14)$$

where C_B is the cost function calculated from one mini-batch.

SGD makes each update computationally cheaper, but the gradient becomes more noisy because only part of the training data is used. The performance therefore depends on the learning rate, batch size, and number of epochs.

3.6 g) Lasso

Lasso regression introduces an L_1 regularization term to the OLS cost function. The cost function used in this project is

$$C(\theta) = \frac{1}{n} \|X\theta - y\|^2 + \frac{\lambda}{n} \|\theta\|_1 \quad (15)$$

The L_1 penalty is defined as

$$\|\theta\|_1 = \sum_j |\theta_j| \quad (16)$$

and since we want to find the gradient of C , let's assume

$$f(\theta_j) = \theta_j$$

then

$$\frac{df}{d\theta_j} = \begin{cases} 1 & \theta_j > 0 \\ 0 & \theta_j = 0 \\ -1 & \theta_j < 0 \end{cases}$$

Unlike Ridge regression, Lasso can make some parameter values exactly zero or close to zero. The absolute value function is not differentiable at zero. Away from zero, the gradient can be written as

$$\nabla C(\theta) = \frac{2}{n} X^T (X\theta - y) + \frac{\lambda}{n} \text{sign}(\theta) \quad (17)$$

At zero, the derivative of the absolute value is not uniquely defined. Instead, the subgradient is the interval $[-1, 1]$.

In our implementation, we use $\text{sign}(\theta)$ for the L_1 term. At $\theta = 0$, PyTorch gives $\text{sign}(0) = 0$. Since $0 \in [-1, 1]$, this is a valid choice of subgradient.

However, subgradient-based optimization does not necessarily produce exactly zero coefficients. The coefficients can instead remain close to zero. Since the Runge function is symmetric about zero, the coefficients corresponding to odd powers are expected to be zero or close to zero.

3.7 d) Cross-validation

Cross-validation is used to estimate how well a model generalizes to unseen data. In k -fold cross-validation, the data are divided into k subsets. The model is trained on $k - 1$ subsets and evaluated on the remaining subset. This procedure is repeated until every subset has been used for validation.

The average validation MSE is

$$\text{CV MSE} = \frac{1}{k} \sum_{i=1}^k \text{MSE}_i \quad (18)$$

In this project, cross-validation is used to compare polynomial degrees and to select regularization parameters for Ridge and Lasso regression.

For Ridge and Lasso regression, feature scaling was performed separately inside each cross-validation fold. The mean and standard deviation were calculated only from the training part of the fold and were then applied to the corresponding validation data. This prevents information from the validation fold from entering the preprocessing step.

3.8 c) Bias-Variance Trade-off

The expected prediction error can be decomposed into bias, variance, and irreducible noise. The decomposition can be written as

$$E[(y - \tilde{y})^2] = \text{Bias}[\tilde{y}]^2 + \text{Var}[\tilde{y}] + \sigma^2. \quad (19)$$

To show this decomposition, we assume that the observed data are generated by

$$y = f(x) + \epsilon,$$

where the noise has mean zero and variance σ^2

$$E[\epsilon] = 0, \quad E[\epsilon^2] = \sigma^2$$

The expected squared prediction error can then be written as

$$E[(y - \tilde{y})^2] = E[(f + \epsilon - \tilde{y})^2]$$

We add and subtract the average of the predictions $E[\tilde{y}]$, and $\tilde{y}$ the predictions,

$$f + \epsilon - \tilde{y} = (f - E[\tilde{y}]) + (E[\tilde{y}] - \tilde{y}) + \epsilon$$

Squaring this expression and taking the expectation yields

$$E[(y - \tilde{y})^2] = (f - E[\tilde{y}])^2 + E[(\tilde{y} - E[\tilde{y}])^2] + E[\epsilon^2] \quad (20)$$

The cross terms disappear because

$$E[\tilde{y} - E[\tilde{y}]] = 0 \quad E[\epsilon] = 0$$

Therefore,

$$E[(y - \tilde{y})^2] = \text{Bias}[\tilde{y}]^2 + \text{Var}[\tilde{y}] + \sigma^2$$

where

$$\text{Bias}[\tilde{y}]^2 = (f - E[\tilde{y}])^2$$

and

$$\text{Var}[\tilde{y}] = E[(\tilde{y} - E[\tilde{y}])^2]$$

The bias measures the difference between the average model prediction and the underlying function, while the variance measures how much the predictions change when different training samples are used.

In practice, the true function $f(x)$ is normally unknown. Therefore, when estimating the bias from data, $f(x)$ can be replaced by the noisy observations y . In this case, the measured bias term also contains the noise variance σ^2 . In our experiment, $\sigma = 0.1$, and therefore $\sigma^2 = 0.01$.

Models with low polynomial degree can have high bias because they are not flexible enough to represent the underlying function. Increasing the polynomial degree can reduce the bias, but it can also increase the variance and lead to overfitting. This gives the bias-variance trade-off.

In this project, bootstrap resampling is used to investigate how bias and variance change with polynomial degree and sample size.

4 Methodology

4.1 Dataset

The data were generated from the Runge function,

$$f(x) = \frac{1}{1 + 25x^2} \quad (21)$$

on the interval $[-1, 1]$. We used $n = 100$ equally spaced data points and added Gaussian noise with standard deviation $\sigma = 0.1$. A random seed of 2026 was used to make the results reproducible.

The noisy observations were generated as

$$y_i = f(x_i) + \epsilon_i \quad (22)$$

where ϵ_i represents Gaussian noise. For the initial OLS and Ridge experiments, the data were divided into 80% training data and 20% test data. For the later experiments, including the bias-variance analysis, gradient descent, Lasso, and SGD, a 70% training and 30% test split was used. The training set was used to fit the regression models, while the test set was used to evaluate their generalization performance.

4.2 Polynomial Design Matrix

For a polynomial of degree p , the design matrix was constructed as a Vandermonde matrix,

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^p \\ 1 & x_2 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{bmatrix} \quad (23)$$

The Vandermonde matrix was used because the polynomial regression represents the model as a linear combination of powers of x . The columns therefore correspond to $x^0, x^1, \dots, x^p$, where $x^0 = 1$ represents the intercept term. Polynomial models with different degrees were tested in order to study underfitting, overfitting, and the effect of model complexity.

4.3 Ordinary Least Squares

Ordinary Least Squares (OLS) is used to find the parameter vector θ that minimizes the squared error between the predicted values and the true values. The prediction is given by

$$\hat{y} = X\theta \quad (24)$$

where X is the design matrix and θ contains the parameters of the polynomial model.

The OLS solution can be written as

$$\theta = (X^T X)^{-1} X^T y \quad (25)$$

However, directly computing the inverse can be numerically unstable if $X^T X$ is singular or close to singular. This can especially happen for high-degree polynomial models. Therefore, in our implementation we used the pseudoinverse,

$$\theta = X^+ y \quad (26)$$

which can also be used when the matrix is not invertible and generally gives a more numerically stable least-squares solution.

After finding θ , the predictions were calculated using

$$\hat{y}_{train} = X_{train}\theta \quad \hat{y}_{test} = X_{test}\theta \quad (27)$$

The prediction error was measured using the Mean Squared Error (MSE),

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (28)$$

OLS minimizes the squared prediction error without directly penalizing the size of the parameters θ .

4.4 Ridge Regression

Ridge regression is similar to OLS, but it adds a regularization term to the cost function. The Ridge cost function is

$$J(\theta) = \|X\theta - y\|_2^2 + \lambda \|\theta\|_2^2 \quad (29)$$

where λ is the regularization parameter.

The first term measures the squared prediction error, while the second term penalizes large values of the parameters θ . When λ is small, Ridge behaves similarly to OLS. When λ becomes larger, the coefficients are pushed toward smaller values. This can reduce overfitting and improve the stability of high-degree polynomial models.

Before applying Ridge regression, the polynomial features were scaled using the mean and standard deviation calculated from the training data. The intercept column was removed and the target values were centered around

their training mean. Therefore, the intercept was not included in the regularization.

Ridge regression was implemented using Singular Value Decomposition (SVD), which avoids directly computing a matrix inverse and improves numerical stability.

Different values of λ were tested to study the effect of regularization on the training and test errors. The values were chosen on a logarithmic scale from 10^{-8} to 10^4 . The MSE and R^2 scores were then used to evaluate the model performance.

4.5 Gradient Descent Optimizers

In this part, several gradient-based optimization methods were implemented and compared: Plain Gradient Descent, Momentum, AdaGrad, RMSprop, and Adam. At each iteration, the gradient of the cost function is computed and used to update the parameter vector θ .

4.5.1 Plain Gradient Descent

Plain Gradient Descent first computes the gradient at the current parameter value,

$$g_t = \nabla J(\theta_t) \quad (30)$$

and updates the parameters using

$$\theta_{t+1} = \theta_t - \eta g_t \quad (31)$$

where θ_t is the current parameter vector, g_t is the current gradient, and η is the learning rate.

Plain Gradient Descent only uses the gradient from the current iteration. It does not use information from previous updates. Therefore, if the loss surface is narrow or has different curvature in different directions, the optimization path can oscillate and convergence can become slow.

4.5.2 Momentum

Momentum extends Gradient Descent by including information from previous gradients. The velocity is initialized as

$$v_0 = 0 \quad (32)$$

and updated using

$$v_t = \beta v_{t-1} + g_t \quad (33)$$

The parameters are then updated by

$$\theta_{t+1} = \theta_t - \eta v_t \quad (34)$$

In our implementation, $\beta = 0.9$ was used. Momentum can be viewed as Gradient Descent with memory. Previous gradients influence the current update, which can reduce oscillations and accelerate movement in directions where the gradients are consistent.

4.5.3 AdaGrad

AdaGrad uses an adaptive learning rate for each parameter. In ordinary Gradient Descent, all parameters use the same learning rate η . AdaGrad instead accumulates the squared gradients,

$$G_t = G_{t-1} + g_t^2 \quad (35)$$

where the square is taken element-wise.

The parameter update is

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{G_t} + \epsilon} \quad (36)$$

If one parameter frequently has large gradients, its accumulated value in G_t becomes large. Therefore, its effective learning rate becomes smaller automatically.

The advantage of AdaGrad is that different parameters can have different effective learning rates. However, since G_t keeps increasing during training, the effective learning rate can eventually become very small. This can make the optimization converge too slowly.

4.5.4 RMSprop

RMSprop tries to solve the problem of the continuously decreasing learning rate in AdaGrad. Instead of accumulating all previous squared gradients, RMSprop uses an exponential moving average,

$$s_t = \rho s_{t-1} + (1 - \rho)g_t^2 \quad (37)$$

The parameter update is

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon} \quad (38)$$

In our implementation, $\rho = 0.9$ was used.

Older squared gradients receive progressively smaller weights. Therefore, RMSprop mainly depends on recent gradient information instead of remembering the complete history. This prevents the effective learning rate from decreasing as aggressively as in AdaGrad.

4.5.5 Adam

Adam, or Adaptive Moment Estimation, combines ideas from Momentum and RMSprop. It keeps two moving averages.

The first moment is

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t \quad (39)$$

which represents a moving average of the gradients.

The second moment is

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2 \quad (40)$$

which represents a moving average of the squared gradients.

Because both moving averages are initialized at zero, bias correction is applied

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (41)$$

and

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (42)$$

The parameter update is then

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (43)$$

In our implementation, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$ were used.

Adam therefore combines the memory of Momentum with the adaptive learning rate of RMSprop.

5 Results

5.1 OLS and Ridge Regression

When we increase the polynomial degree, the training MSE becomes smaller. However, after around degree 13, the test MSE starts to increase. This means that the model begins to overfit. The model fits the training data better, but the prediction on new data becomes worse.

We can see the same behavior from the R^2 score. The training R^2 increases when the degree becomes higher, but the test R^2 starts to decrease for very high degrees. Therefore, a higher polynomial degree does not always give a better model.

For OLS, the coefficients can also become very large when the degree is high. This makes the model unstable and very sensitive to the data. The problem is even stronger when we only have a small number of data points. For high degrees, the test MSE can become very large and the test R^2 can become very negative.

When we use more data points, the model becomes more stable. The difference between the training and test results also becomes smaller. This means that high-degree polynomial models need more data to work well.

We also tested different noise levels. When σ increases, both training and test MSE increase, while the R^2 score becomes smaller. This is expected because more noise makes the Runge function harder to learn.

For Ridge regression, we also have the regularization parameter λ . When λ is very small, Ridge gives almost the same result as OLS. When λ increases, the coefficients become smaller because large coefficients are penalized.

This makes Ridge more stable than OLS for high polynomial degrees. However, if λ becomes too large, the model is regularized too much and starts to underfit.

So the choice of λ is important. A small λ gives weak regularization, while a large λ gives strong regularization. For moderate polynomial degrees, both OLS and Ridge can give good results. For high degrees, Ridge is usually more stable.

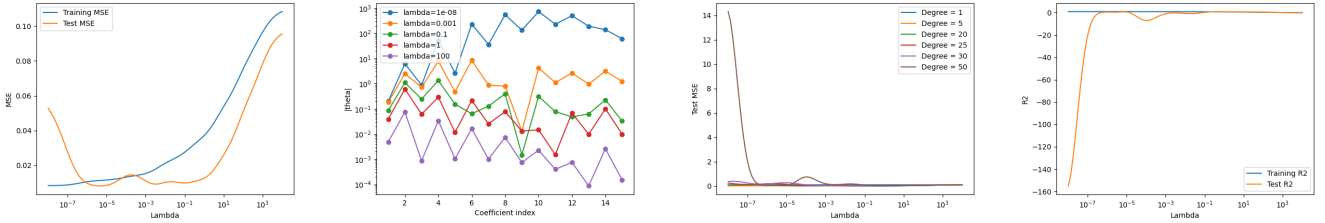


Figure 1: Ridge regression becomes more stable than OLS at high polynomial degrees because regularization reduces large coefficients, while excessively large values of λ lead to underfitting.

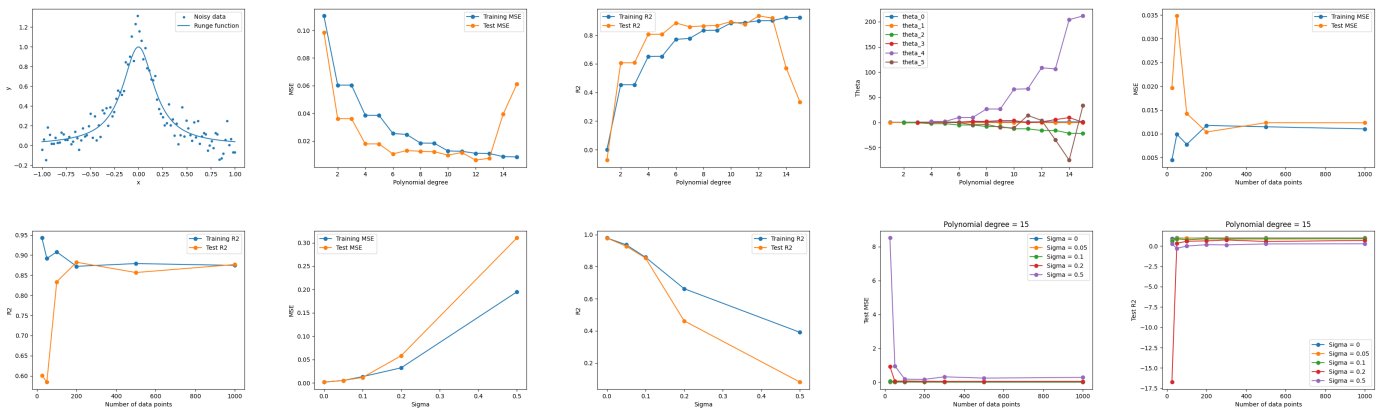


Figure 2: OLS performs well at moderate polynomial degrees, but the test error increases when the degree becomes too large. Increasing the number of data points improves stability, while increasing the noise level reduces prediction accuracy.

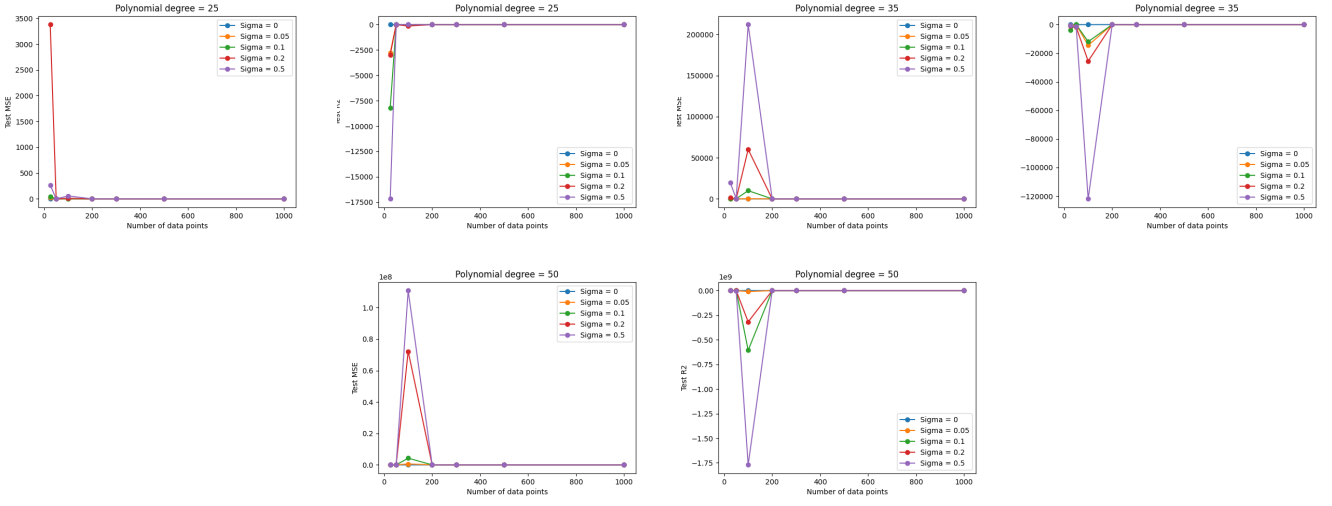


Figure 3: At high polynomial degrees, the OLS test error increases strongly while the training error remains low, showing overfitting and numerical instability.

5.2 Bias-Variance Analysis

We studied the bias-variance trade-off using bootstrap. We used polynomial degrees from 1 to 30 and 100 bootstrap samples for each degree. The data were split into 70% training data and 30% test data.

For small data sets, especially $n = 50$ and $n = 100$, the model becomes unstable when the polynomial degree is high. For low degrees, the bias squared is relatively large because the model is too simple. When the degree increases, the bias first becomes smaller.

However, for high polynomial degrees, the variance becomes very large. For $n = 50$ and $n = 100$, both the variance and test error increase a lot. This means that the model is very sensitive to changes in the training data and starts to overfit.

When we increase the number of data points to $n = 500$ and $n = 1000$, the variance becomes smaller and the model becomes more stable. The test error also stays low for a larger range of polynomial degrees.

For $n = 10000$, the variance is very small compared with the bias squared. The test error is then very close to the measured bias squared for most polynomial degrees. This shows that more training data can reduce the variance.

In our implementation, the bias squared is calculated using the noisy test values y instead of the true function $f(x)$. Because of this, the measured bias also contains the noise variance. We used $\sigma = 0.1$, so $\sigma^2 = 0.01$. This is why the bias squared becomes close to 10^{-2} for the largest data set.

Since the true Runge function is known in this experiment, we could also calculate the bias using the true $f(x)$. In that case, the noise would not be included in the bias term. Here, we keep the version based on the noisy values y .

The training and test MSE also show overfitting. For low and moderate degrees, the two errors are quite similar. For very high degrees, the test MSE becomes much larger while the training MSE stays small.

So the main result is that low-degree models have larger bias, while high-degree models can have large variance. Increasing the number of data points reduces the variance and makes the model more stable.

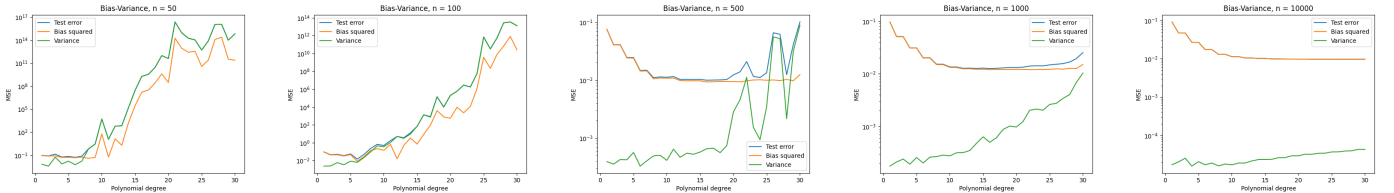


Figure 4: For small data sets ($n = 50$ and $n = 100$), the variance and test error increase strongly at high polynomial degrees, while for $n = 10000$ the variance is much smaller and the model remains stable over a larger range of degrees.

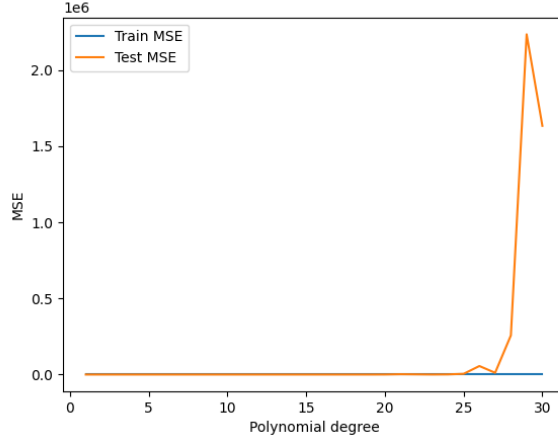


Figure 5: The training and test MSE are similar for low and moderate polynomial degrees, while the test MSE increases strongly at high degrees as the model starts to overfit.

5.3 Cross-validation

We used k -fold cross-validation to test OLS and Ridge regression. We used both $k = 5$ and $k = 10$, and tested polynomial degrees from 1 to 30.

For OLS, the cross-validation MSE first becomes smaller when the polynomial degree increases. The lowest MSE is around degree 11 for both 5-fold and 10-fold cross-validation. After degree 11, the error starts to increase. For very high degrees, the error becomes very large. This means that the model starts to overfit and becomes unstable.

The 5-fold and 10-fold results are very similar near the minimum. Both methods select almost the same model complexity. For very high degrees, the 5-fold result becomes more unstable, while the 10-fold curve is a little smoother.

For Ridge regression, we tested different values of λ from 10^{-8} to 10^4 . For each polynomial degree, we selected the λ that gave the lowest cross-validation MSE.

When λ is very small, Ridge behaves almost like OLS. If λ becomes too large, the error increases because the model is regularized too much. The effect of λ also becomes more important for higher polynomial degrees.

For low and moderate degrees, OLS and Ridge give similar results. However, for high degrees, the OLS error becomes very large while Ridge stays much more stable. This is because Ridge keeps the coefficients smaller.

Near the best polynomial degree, the best λ for Ridge is very small. Because of this, Ridge and OLS give almost the same minimum cross-validation MSE.

Both bootstrap and cross-validation give the same main result. Moderate polynomial degrees work well, while very high degrees become unstable. Cross-validation gives a more clear model choice, and both 5-fold and 10-fold cross-validation select degree 11 for OLS.

The exact best degree from bootstrap and cross-validation does not need to be exactly the same because the two methods use different resampling procedures. The important point is that both methods show the same general behavior.

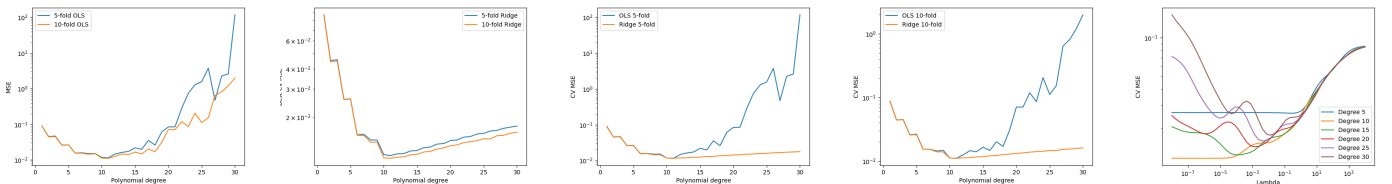


Figure 6: Both 5-fold and 10-fold cross-validation select a polynomial degree around 11 for OLS and Ridge. OLS becomes unstable at high degrees, while Ridge remains more stable because regularization controls the growth of the coefficients.

5.4 Gradient Descent

We implemented Gradient Descent for both OLS and Ridge regression. We calculated the gradients analytically and also used PyTorch automatic differentiation to check the results.

For OLS, the analytical gradient is

$$\nabla C(\theta) = \frac{2}{n} X^T (X\theta - y)$$

The analytical gradient and the PyTorch gradient are almost the same. The norm of the difference is about 3.97×10^{-18} . This shows that our analytical gradient is implemented correctly.

Automatic differentiation is different from finite-difference methods. PyTorch follows the operations in the computation and uses the chain rule to calculate the gradient.

For OLS Gradient Descent, we used a learning rate of $\eta = 0.01$ and 10000 iterations. The training MSE decreases during the optimization, so Gradient Descent moves toward the minimum. However, after 10000 iterations, the parameters are still different from the closed-form solution. The norm of the difference is about 2.17. This means that the method is stable, but the convergence is slow.

The largest eigenvalue of the OLS Hessian is about 2.304. The maximum stable learning rate is approximately

$$\eta_{\max} \approx \frac{2}{\lambda_{\max}} \approx 0.868$$

Since we used $\eta = 0.01$, the learning rate is much smaller than this limit. This makes the optimization stable, but it can also make the convergence slow.

The predictions from Gradient Descent and the closed-form solution are quite similar, even when the parameter values are different. This can happen because high-degree polynomial regression can be badly conditioned.

For Ridge regression, the gradient is

$$\nabla C(\theta) = \frac{2}{n} (X^T (X\theta - y) + \lambda\theta)$$

The analytical Ridge gradient also agrees very well with the PyTorch gradient. The norm of the difference is about 3.89×10^{-14} .

For low polynomial degrees, Gradient Descent gives results close to the closed-form Ridge solution. For degree 1, the difference is about 4.3×10^{-17} , and for degree 5 it is about 0.124. For degree 20 and higher, the difference becomes much larger.

This means that using the same learning rate and the same number of iterations does not work equally well for every polynomial degree. High-degree polynomial models are harder to optimize because the design matrix becomes more ill-conditioned.

For the high-degree Ridge model, the largest eigenvalue of the Hessian is about 85.68, which gives a maximum stable learning rate of about 0.0233. Our learning rate $\eta = 0.01$ is below this value, so the optimization is still stable, but it can converge slowly.

In general, the analytical and automatic gradients give almost the same result. Gradient Descent reduces the loss, but the convergence depends on the learning rate, number of iterations, and polynomial degree.

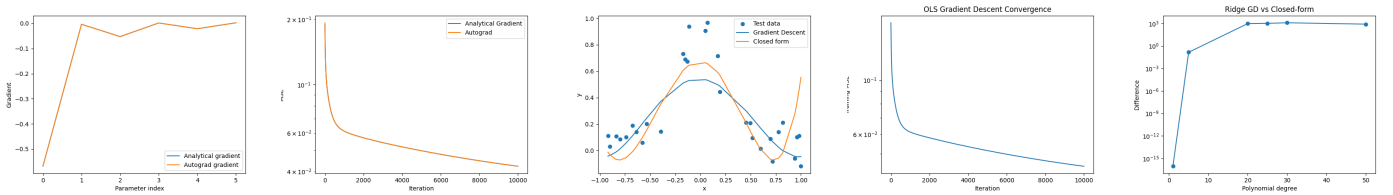


Figure 7: The analytical and automatic gradients agree to approximately 10^{-18} for OLS and 10^{-14} for Ridge. Gradient Descent reduces the training loss, but convergence becomes much slower for high-degree polynomial models because the problem becomes increasingly ill-conditioned.

5.5 Optimization Methods

We compared Plain Gradient Descent, Momentum, AdaGrad, RMSprop, and Adam for both OLS and Ridge regression. We compared how the training MSE changes with the number of iterations.

For OLS, Plain Gradient Descent converges slowly. After 10000 iterations, the training MSE is still higher than for several of the other methods. AdaGrad improves faster at the beginning, but later the convergence becomes slower. Momentum converges faster than Plain Gradient Descent.

RMSprop and Adam reduce the training MSE very quickly. They reach a low training error much earlier than Plain Gradient Descent and AdaGrad. We see almost the same behavior for Ridge regression. Momentum improves the convergence, while RMSprop and Adam reach the low-error region faster.

We also tested different learning rates for Adam. We used $\eta = 0.001$, $\eta = 0.01$, and $\eta = 0.05$.

For OLS, $\eta = 0.001$ is stable but slow. With $\eta = 0.01$, Adam converges much faster and is still stable. When we increase the learning rate to $\eta = 0.05$, the loss decreases very quickly at first, but it starts to oscillate near the minimum.

The Ridge results show the same behavior. A learning rate of 0.001 is slow, while 0.01 converges faster. With $\eta = 0.05$, the loss reaches a low value quickly, but we can see several spikes during the optimization.

From these results, Momentum is faster than Plain Gradient Descent in our experiments. RMSprop and Adam also converge quickly. For Adam, $\eta = 0.01$ works better than $\eta = 0.001$, while $\eta = 0.05$ is less stable near the minimum.

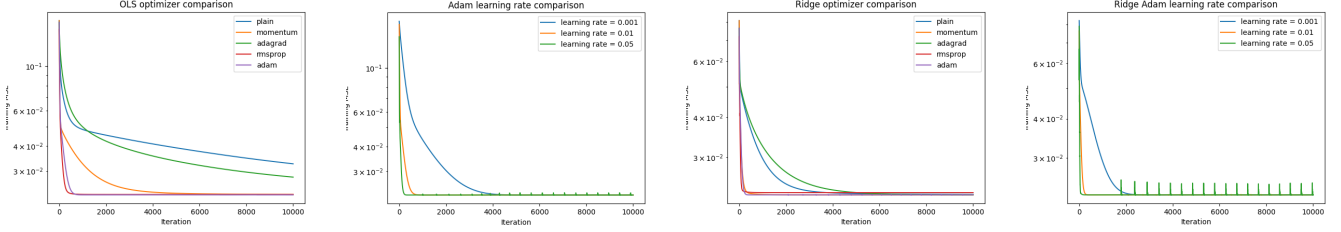


Figure 8: Momentum converges faster than Plain Gradient Descent, while RMSprop and Adam reach a low training error more rapidly for both OLS and Ridge. For Adam, increasing the learning rate from 0.001 to 0.01 improves convergence speed, while $\eta = 0.05$ introduces oscillations near the minimum.

5.6 Lasso Regression

We implemented Lasso regression with the L_1 penalty and compared Plain Gradient Descent, Momentum, AdaGrad, RMSprop, and Adam. We used a learning rate of 0.01 and 10000 iterations.

The optimization method has a clear effect on the convergence speed. Plain Gradient Descent converges slowly. AdaGrad improves faster at the beginning, while Momentum converges faster than both Plain Gradient Descent and AdaGrad. RMSprop and Adam reach a low training error very quickly.

However, the fastest method does not always give the lowest test error. For $\lambda = 0.01$, AdaGrad gives the lowest test MSE among the tested methods. The test MSE is about 0.0243, and the R^2 score is about 0.722.

Plain Gradient Descent gives a test MSE of about 0.0278. Momentum, RMSprop, and Adam give about 0.0359, 0.0407, and 0.0396. So even though RMSprop and Adam converge faster on the training loss, AdaGrad gives the best test result in this experiment.

We also tested different values of λ . For small values, from 10^{-4} to 10^{-1} , the test MSE changes only a little and stays close to 0.024. The lowest test MSE is about 0.0240 at $\lambda = 10^{-4}$.

When λ is increased to 1, the test MSE becomes about 0.0425. This means that too much regularization makes the model underfit.

The number of coefficients close to zero also changes with λ . For very small λ , no coefficients are close to zero. At $\lambda = 0.01$, one coefficient is close to zero. At $\lambda = 1$, four coefficients are close to zero. This shows that stronger L_1 regularization can make the model more sparse. We also compared OLS, Ridge, and Lasso using the test MSE for the tested settings. OLS gives about 0.0430, Ridge gives about 0.0366, and Lasso gives about 0.0243. The corresponding R^2 scores are about 0.508, 0.581, and 0.722.

So in this experiment, Lasso gives the lowest test MSE for these settings. We also see that a method can converge faster on the training loss without giving the best result on the test data.

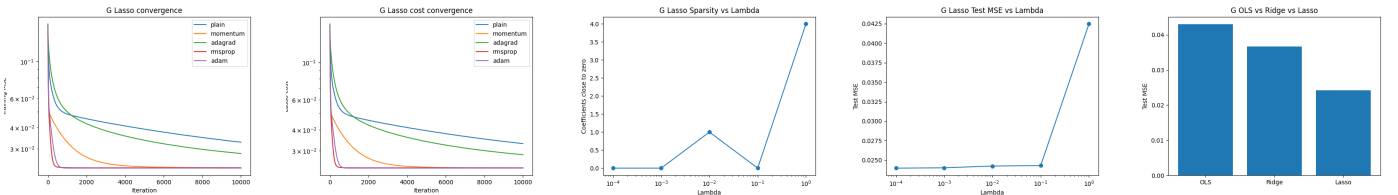


Figure 9: For $\lambda = 0.01$, AdaGrad gives the lowest Lasso test MSE of approximately 0.0243, while RMSprop and Adam converge faster on the training loss. Increasing λ increases coefficient sparsity, but strong regularization also increases the test error.

Stochastic Gradient Descent

We implemented Stochastic Gradient Descent using mini-batches. We studied the effect of batch size, number of epochs, and learning-rate schedule. We also compared mini-batch SGD with full-batch Gradient Descent.

The batch size affects the test MSE. In our experiment, a small batch size gives a lower test error. When the batch size becomes large, the result becomes more similar to full-batch Gradient Descent.

The number of epochs is also important. With only a few epochs, the model has not converged and the test MSE is high. When we increase the number of epochs, the test MSE becomes smaller. In our result, the test MSE decreases from about 0.095 at 10 epochs to about 0.023 at 5000 epochs.

We also tested a learning-rate schedule. The test MSE decreases quickly at the beginning and then more slowly later. The learning rate becomes smaller during training, so the updates are also smaller when the model gets closer to the minimum. After 5000 epochs, the test MSE is about 0.032.

We then compared mini-batch SGD with full-batch Gradient Descent. After 5000 epochs, mini-batch SGD gives a test MSE of about 0.032, while full-batch Gradient Descent gives about 0.037. In our experiment, mini-batch SGD therefore gives a slightly lower test error.

The fitted model follows the main shape of the data, but it does not fit every noisy test point. This is expected because the data contain noise.

For full-batch Gradient Descent, all training data are used for every update. Mini-batch SGD only uses part of the training data for each update, so one update is cheaper. However, SGD makes more updates and the gradient is more noisy. For our small data set, the difference in computational cost is not very large.

So the SGD result depends on the batch size, number of epochs, and learning rate. A small batch size and enough epochs give better results in our experiment.

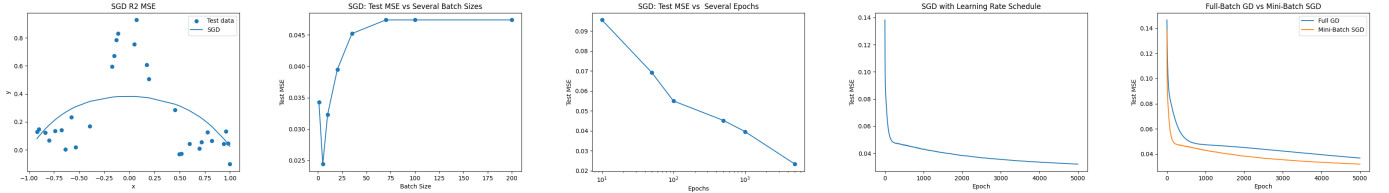


Figure 10: Mini-batch SGD performs best with a relatively small batch size and enough training epochs. Increasing the number of epochs reduces the test MSE substantially, and a decreasing learning-rate schedule gives stable convergence. In our experiment, mini-batch SGD reaches a slightly lower final test MSE than full-batch Gradient Descent.

5.7 Final model comparisons

For the final model comparison, we used 5-fold and 10-fold cross-validation for OLS, Ridge, and Lasso regression. The polynomial degree was varied, while the regularization parameter was selected for Ridge and Lasso based on the lowest cross-validation MSE and corresponding standard error (SE)

Model	k	Best degree	Best λ	CV MSE	SE
OLS	5	11	—	0.01154	0.00142
OLS	10	11	—	0.01111	0.00161
Ridge	5	11	1.0×10^{-8}	0.01154	0.00142
Ridge	10	11	8.69×10^{-6}	0.01110	0.00162
Lasso	5	8	2.15×10^{-3}	0.01972	0.00349
Lasso	10	8	1.0×10^{-4}	0.01963	0.00295

Table 1: Best cross-validation results for OLS, Ridge, and Lasso regression.

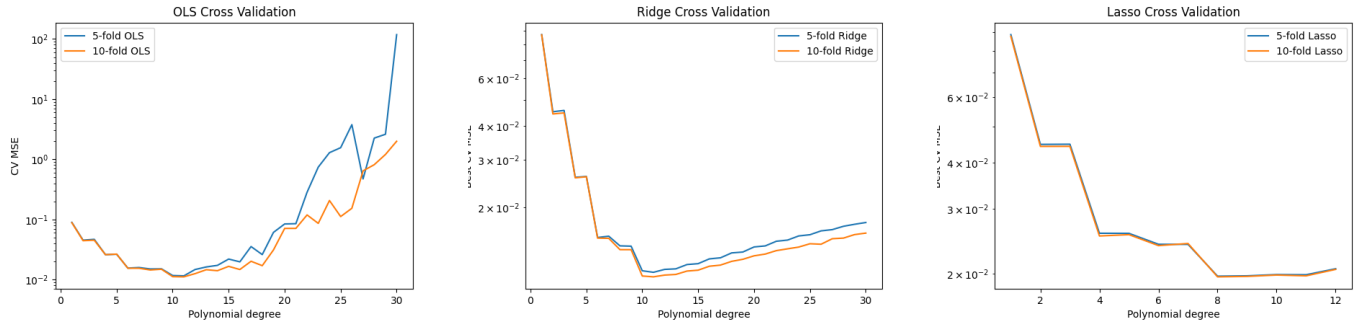


Figure 11: Cross-validation selects degree 11 for OLS and Ridge and degree 8 for Lasso. The minimum 10-fold CV MSE is approximately 0.01111 for OLS, 0.01110 for Ridge, and 0.01963 for Lasso. Ridge remains considerably more stable than OLS at high polynomial degrees.

For OLS, both 5-fold and 10-fold cross-validation select degree 11. The lowest CV MSE is about 0.01154 for 5-fold and 0.01111 for 10-fold cross-validation. When the polynomial degree becomes larger, the OLS error increases a lot. This shows that OLS becomes unstable for very high degrees.

Ridge also selects degree 11. For 5-fold cross-validation, the best λ is 10^{-8} and the CV MSE is about 0.01154. For 10-fold cross-validation, the best λ is about 8.69×10^{-6} and the CV MSE is about 0.01110.

The best values of λ are very small, so Ridge behaves almost like OLS near the minimum. However, for higher polynomial degrees, Ridge is more stable than OLS.

For Lasso, both 5-fold and 10-fold cross-validation select degree 8. The best λ is about 2.15×10^{-3} for 5-fold and 10^{-4} for 10-fold cross-validation. The CV MSE is about 0.01972 and 0.01963, respectively. These values are higher than the results for OLS and Ridge.

For 10-fold cross-validation, Ridge gives an MSE of about 0.01110, while OLS gives about 0.01111. The difference is very small compared with the standard errors, which are both around 0.0016. So OLS and Ridge give almost the same result near the minimum.

Ridge has the lowest mean CV MSE among the tested models, but the difference from OLS is very small. Lasso gives a higher CV MSE, but it selects a lower polynomial degree and also shrinks the coefficients through the L_1 penalty.

6 Discussion

The results show that polynomial degree, number of data points, noise, and regularization all affect the model performance. For low polynomial degrees, the model is too simple and the bias is high. When the degree increases, the model can fit the Runge function better and the prediction error becomes smaller. However, if the degree becomes too high, the variance increases and the model can overfit the training data. The model can also become numerically unstable.

The bias-variance results show the same behavior. For small data sets, high-degree polynomial models have very large variance. When we increase the number of data points, the variance becomes smaller and the model becomes more stable. This means that more training data can make high-degree polynomial models less sensitive to the training set.

The comparison between OLS and Ridge also shows the effect of regularization. OLS works well for moderate polynomial degrees, but the coefficients can become very large for high degrees. Ridge reduces the size of the coefficients and makes the model more stable. However, if λ is too large, the model is regularized too much and starts to underfit.

Lasso uses an L_1 penalty instead of the L_2 penalty used by Ridge. When λ increases, more coefficients can become close to zero. This can give a simpler model. However, if the regularization is too strong, the test error increases. Therefore, the choice of λ is important for both Ridge and Lasso.

We used cross-validation to choose the polynomial degree and the regularization parameter. Both 5-fold and 10-fold cross-validation selected degree 11 for OLS and Ridge, while Lasso selected degree 8. For 10-fold cross-

validation, OLS gave a CV MSE of about 0.01111 and Ridge gave about 0.01110. The difference is very small compared with the standard errors, so OLS and Ridge give almost the same result near the minimum. Lasso gave a higher CV MSE of about 0.01963.

We also compared different optimization methods. Plain Gradient Descent converges slowly in our experiments. Momentum converges faster because it also uses information from previous updates. RMSprop and Adam reach a low training error quickly. However, faster convergence does not always give the lowest test error. For Lasso, AdaGrad gave a lower test MSE than RMSprop and Adam in our experiment.

Our optimizer comparison is mainly based on the convergence curves and a fixed number of iterations. This is not a completely fair comparison of computational efficiency. Another way would be to compare how many iterations each method needs to reach the same loss value. Therefore, the method that converges fastest in our plots is not always the best method for every situation.

The SGD results show that batch size, number of epochs, and learning rate all affect the result. Small mini-batches make each update cheaper, but the gradient is also more noisy. For our small data set, the computational advantage is not very large. SGD becomes more useful when the data set is larger.

There are also some limitations in this project. The Runge function is a simple one-dimensional test problem, so the same results may not hold for more complicated data sets. The selected polynomial degree and regularization parameter can also change with the data split and random seed. However, the experiments still show the main ideas of overfitting, regularization, bias-variance trade-off, cross-validation, and gradient descent methods.

7 Conclusion

We have studied polynomial regression for the Runge function using OLS, Ridge and Lasso regression. We also applied different methods like gradient descent, bootstrap, cross-validation, and Stochastic Gradient Descent.

The polynomial degree has a large effect on the model performance. When the degree increases, the model can fit the Runge function better. However, if the degree becomes too high, the model can overfit and become unstable. We also found that more data reduce the variance and make the model more stable. Ridge reduces large coefficients, while Lasso can push some coefficients close to zero.

Cross-validation selected degree 11 for OLS and Ridge, while Lasso selected degree 8. For 10-fold cross-validation, Ridge gave an MSE of about 0.01110, while OLS gave about 0.01111. The difference is very small compared with the standard errors, so OLS and Ridge give almost the same result around the minimum. Lasso gave a higher MSE of about 0.01963.

We also compared different optimization methods. Momentum converges faster than Plain Gradient Descent in our experiments. RMSprop and Adam also reach a low training error quickly. AdaGrad can become slower later because its learning rate becomes smaller. We also found that faster convergence does not always give the lowest test error.

For SGD, the result depends on the batch size, number of epochs, and learning rate. Mini-batch SGD uses only part of the data for each update, so each update can be cheaper than full-batch Gradient Descent.

From these experiments, we can see that a more complex model does not always give a better result. The polynomial degree and regularization both affect the prediction error and model stability. Cross-validation is useful for choosing the polynomial degree and the regularization parameter. The optimization method is also important because different methods can have very different convergence speeds.

A Use of AI/LLM Tools and references

A.1 Tools Used

ChatGPT (OpenAI, accessed through ChatGPT in September 2026) was used during the project as an aid for writing, code debugging, and discussion of the numerical results. Additional source used was GeeksforGeeks.

A.2 Text Writing and Editing

Section	Use of LLM
Theory	Some section formulas
Methodology	Grammar
Results	Structuring and explaining numerical results
Discussion	Organizing results
Conclusion	Assistance with structure and language

Table 2: LLM assistance used in the written report.

ChatGPT was used to assist with the structure and language of the conclusion. The conclusions themselves were based on the numerical results obtained from our own implementation. All reported values, figures, and interpretations were checked against our results.

A.3 Code Generation and Assistance

Code	Use of LLM
Bias-variance code	Debugging
Cross-validation code	Debugging
Gradient descent code	Debugging
Optimizer comparison	Debugging
SGD code	Implementation and debugging
Final model comparison	CV output, standard errors, and debugging

Table 3: LLM assistance used in the project code.

ChatGPT was used to assist with debugging, code organization, and small changes to parts of the implementation. The code was integrated into our own implementation and tested using the numerical experiments in the report. Where analytical or closed-form solutions were available, these were used to check the numerical results.

We take responsibility for all submitted code, results, interpretations, and conclusions.

A.4 Code availability

The source code used in this project is available in the following GitHub repository:

<https://github.com/zejingw1998/FYS-STK4155-Anvendt-dataanalyse-og-maskinl-ring>

B References

- [1] PyTorch, *PyTorch Documentation*. Available at: <https://pytorch.org/docs/stable/>
- [2] Scikit-learn, *Scikit-learn Documentation*. Available at: <https://scikit-learn.org/stable/>
- [3] FYS-STK4155, *Applied Data Analysis and Machine Learning: Lecture Notes*, University of Oslo, 2026.